Curs 5B

Clase Abstracte si Interfete

Programare Orientată pe Obiecte

Clase abstracte

Programare Orientată pe Obiecte

Clase abstracte

- clasele pentru care programatorul nu intenţionează să instanţieze obiecte.
- clase de bază in ierarhii
- se mai numesc ***clase abstracte de bază***
- o clasă devine abstractă dacă conţine una sau mai multe declaraţii de funcţii abstracte
- dacă o clasă ce moşteneşte o clasă ce conţine o funcţie abstractă nu implementează acea funcţie abstractă moştenită, devine ea însăşi o *clasă abstractă*.

OBS: clasele pentru care se pot instanţia obiecte se numesc ***clase concrete***.

Clase şi metode abstracte

Clasă abstractă:

```
[public] abstract class ClasaAbstracta ... {
    // Declaratii uzuale
    // Declaratii de metode abstracte
}
```

Metodă abstractă: doar interfaţa, nu si implementarea

```
abstract class ClasaAbstracta {
    abstract void metodaAbstracta(); // Corect
    void metoda(); // Eroare
}
```

- **O metodă abstractă nu poate apărea decât într-o clasă abstractă!**
- **Orice clasă care are o metodă abstractă trebuie declarată ca fiind abstractă!**

Exemple:

Number: Integer, Double, …

Component: Button, List, ...

Clase abstracte

- **Clasă abstractă:** interfaţa comună, funcţionalitate diferită pentru fiecare subtip, ce anume au clasele derivate în comun.
- **Creăm o clasă abstractă pentru:**
- ✓ **manipularea unui set de clase printr-o interfaţă comună**
- ✓ **reutilizarea unei serii de metode si membri din această clasă in clasele derivate.**
- Metodele suprascrise în clasele derivate vor fi apelate folosind dynamic binding (late binding)!
- O clasă abstractă poate să nu aibă nici o metodă abstractă!
- *Nu se pot crea instanţe ale unei clase abstracte, aceasta exprimând doar un punct de pornire pentru definirea unor instrumente reale!* => crearea unui obiect al unei clase abstracte eroare la compilare
- Ex. Number nr=new Number(); // NU

  Number nr=new Integer(); // DA

Clase abstracte în contextul moştenirii

- O clasă care moşteneşte o clasă abstractă este ea însăşi abstractă daca nu implementează **toate** metodele abstracte ale clasei de bază.

⇒ O clasă care poate fi instanţiată (nu este abstractă) şi care moşteneşte o clasă abstractă trebuie să implementeze toate metodele abstracte pe lanţul moştenirii

- Este posibil să declarăm o **clasă abstractă fără** ca ea să aibă **metode abstracte** - când declarăm o clasă pentru care nu dorim instanţe (nu este corect *conceptual* să avem obiecte de tipul acelei clase, chiar dacă definiţia ei este completă).

Interfeţe

- Ce este o interfaţă ?
- Definirea unei interfeţe
- Implementarea unei interfeţe
- Interfeţe şi clase abstracte
- Moştenire multiplă prin interfeţe
- Utilitatea interfeţelor
- Transmiterea metodelor ca parametri
- Compararea obiectelor

Ce este o interfaţă ?

- **Colecţie de metode abstracte şi declaraţii de constante**

- Defineşte un set de metode dar nu specifică nici o implementare pentru ele.
- Duce conceptul de clasă abstractă cu un pas înainte prin eliminarea oricăror implementări de metode
- Separarea modelului de implementare
- Protocol de comunicare
- O clasă care implementează o interfaţă trebuie obligatoriu să specifice implementări pentru toate metodele interfeţei, supunându-se aşadar unui anumit comportament.
- Defineşte noi tipuri de date
- Clasele pot implementa interfeţe

Definirea unei interfeţe

```
[public] interface NumeInterfata
[extends SuperInterfata1, SuperInterfata2...]
{
    /* Corpul interfetei:
    Declaraţii de constante
    Declaraţii de metode abstracte
    */
}
```

Corpul unei interfeţe poate conţine:

- constante: acestea pot fi sau nu declarate cu modificatorii **public, static şi final** care sunt **impliciţi**, nici un alt modificator neputând apărea în declaraţia unei variabile dintr-o interfaţă.
  - Constantele unei interfeţe trebuie obligatoriu iniţializate, însă pot fi iniţializate cu **valori neconstante -** vor fi iniţializate la iniţializarea clasei.
- metode fără implementare: acestea pot fi sau nu declarate cu modificatorul **public**, care este **implicit**; nici un alt modificator nu poate apărea în declaraţia unei metode a unei interfeţe.

Definirea unei interfeţe

Atenţie!

- Variabilele unei interfeţe sunt implicit **publice** chiar dacă nu sunt declarate cu modificatorul public.
- Variabilele unei interfeţe sunt implicit **constante** chiar dacă nu sunt declarate cu modificatorii static şi final.
- Metodele unei interfeţe sunt implicit **publice** chiar dacă nu sunt declarate cu modificatorul public.

```
interface Exemplu {
    int MAX = 100;    // echivalent cu:
    public static final int MAX = 100;
    int MAX;   // Incorect, lipseste initializarea
    private int x = 1;  // Incorect, modificator nepermis
    void metoda();    // Echivalent cu:
    public void metoda();
    protected void metoda2();
    // Incorect, modificator nepermis
}
```

Exemplu initializare

- Expresia de iniţializare este evaluată şi asignată exact o dată, atunci când interfaţa este iniţializată.
- O eroare de compilare apare dacă o expresie de iniţializare pentru un câmp conţine o referinţă la un nume de câmp care este declarat mai târziu.
- Exemplu:

```
interfaceTest {
    float f=j;
    int j=1;
    int k=k+1;
}
```

Erori de compilare:

- **j** este referit la iniţializarea lui **f**, înainte ca **j** să fie declarat
- iniţializarea lui **k** se referă la însuşi **k**.

Implementarea unei interfeţe

class NumeClasa implements NumeInterfata

sau:

class NumeClasa implements Interfata1, Interfata2, ...

- O clasă care implementează o interfaţă, pentru a fi instanţiabilă trebuie obligatoriu să specifice cod pentru toate metodele interfeţei.
- O clasă poate avea şi alte metode şi variabile membre în afară de cele definite în interfaţă.
- Implementarea unei interfeţe poate să fie şi o clasă abstractă.
- ***Spunem că un obiect are tipul X, unde X este o interfaţă, dacă acesta este o instanţă a unei clase ce implementează interfaţa X.***
- Atenţie! Modificarea unei interfeţe implică modificarea tuturor claselor care implementează acea interfaţă.

Exemplu: implementarea unei stive (1)

- Interfaţa ce descrie stiva:

```
public interface Stack {
    void push ( Object item ) throws StackException ;
    void pop () throws StackException ;
    Object peek () throws StackException ;
    boolean empty ();
    String toString ();
}
```

- Clasa ce defineşte o excepţie proprie StackException:

```
public class StackException extends Exception {
    public StackException () {
        super (); // putea lipsi !!!
    }
    public StackException ( String msg) {
        super (msg);
    }
}
```

Exemplu: implementarea unei stive folosind un vector

```
public class StackImpl1 implements Stack {
    private Object items [];   // Vect. ce contine ob.
    private int n=0;  // Nr. curent de elem. din stiva
    public StackImpl1 ( int max ) {   // Constructor
        items = new Object [ max ];
    }
    public StackImpl1 () {
        this (100) ;
    }
    public void push ( Object item ) throws
    StackException {
        if (n == items . length )
            throw new StackException (" Stiva e
    plina !");
        items [n++] = item ;
    }
```

Exemplu: implementarea unei stive folosind un vector (2)

```
    public void pop () throws StackException {
        if ( empty ())
            throw new StackException (" Stiva e vida !");
        items [--n] = null ;
    }
    public Object peek () throws StackException {
        if ( empty ())
            throw new StackException (" Stiva e vida !");
        return items [n -1];
    }
    public boolean empty () {
        return n ==0 ;
    }
    public String toString () {
        String s="";
        for (int i=n -1; i >=0; i --)
                s += items [i] + " ";
        return s;
    }
}
```

Exemplu: implementarea unei stive folosind o lista inlantuita (1)

```
public class StackImpl2 implements Stack {
    class Node {  // Clasa interna ce reprezinta un nod al listei
        Object item ; // informatia din nod
        Node link ; // legatura la urmatorul nod
        Node ( Object item , Node link ) {
            this . item = item ;
            this . link = link ;
        }
    }
    private Node top= null ; // Referinta la varful stivei
    public void push ( Object item ) {
        Node node = new Node (item , top);
        top = node ;
    }
    public void pop () throws StackException {
        if ( empty ())
          throw new StackException (" Stiva este vida !");
        top = top . link ;
    }
```

Exemplu: implementarea unei stive folosind o lista inlantuita (2)

```
    public Object peek () throws StackException {
        if ( empty ())
            throw new StackException (" Stiva este vida !");
        return top. item ;
    }
    public boolean empty () {
            return  top == null;
    }
    public String toString () {
            String s="";
            Node node = top;
            while ( node != null ) {
                        s +=  node . item  + " ";
                        node = node . link ;
            }
            return s;
    }
}
```

Observaţii

- Deşi metoda push din interfaţă declară aruncarea unor excepţii de tipul StackException, nu este obligatoriu ca metoda din clasă să specifice şi ea acest lucru, atâta timp cât nu generează excepţii de acel tip!
- Invers este însă obligatoriu!

Folosirea stivei:

```
public class TestStiva {
  public static void afiseaza ( Stack s) {
      System . out. println (" Continutul stivei este : " + s);
  }
  public static void main ( String args []){
            try {
                Stack s1 = new StackImpl1 ();
               //Stack s1 = new Stack();    //f gresit!!!
                 s1. push ("a");
                 s1. push ("b");
         System . out. println ("Continutul stivei este : " + s1);
         // sau afiseaza(s1);
                 Stack s2 = new StackImpl2 ();
                 s2. push ( new Integer (1));
                 s2. push ( new Double (3.14) );
         System . out. println ("Continutul stivei este : " + s2);
        // sau afiseaza(s2);
            } catch ( StackException e) {
               System . err. println (" Eroare la lucrul cu stiva!");
             e. printStackTrace ();
         }
    }
}
```

Interfeţe şi clase abstracte

- “O clasă abstractă nu ar putea înlocui o interfaţă ?”
- Unele clase sunt forţate să extindă o anumită clasă (de exemplu orice applet trebuie să fie subclasa a clasei Applet) şi nu ar mai putea sa extindă o altă clasă. Fără folosirea interfeţelor nu am putea forţa clasa respectivă să respecte diverse tipuri de protocoale
- Extinderea unei clase abstracte forţează o relaţie între clase
- Implementarea unei interfeţe specifică doar necesitatea implementării unor anumite metode
- Interfeţele şi clasele abstracte nu se exclud, fiind folosite “împreună”:
    - •List – interfaţă
    - •AbstractList – clasă abstractă, implementează interfaţa List
    - •LinkedList, ArrayList – clase concrete, instanţiabile derivate din AbstractList!

“Moştenire” multiplă prin interfeţe

- class NumeClasa extends ClasaUnica implements Interfata1, Interfata2, ...
- interface NumeInterfata extends Interfata1, Interfata2, ...
- Ierarhia interfeţelor este independentă de ierarhia claselor care le implementează.

```
interface I1 {
    int a=1;
    void metoda1();
}
interface I2 {
    int b=2;
    void metoda2();
}
class C implements I1, I2 {
    public void metoda1() {...}
    public void metoda2() {...}
}
```

Ambiguităţi

```
interface I1 {
    int x=1;
    void metoda();
}
interface I2 {
    int x=2;
    void metoda(); //corect
    //int metoda(); //incorect
}
class C implements I1, I2 {
    public void metoda() {
            System.out.println(I1.x); //corect
            System.out.println(I2.x); //corect
            System.out.println(x); //ambiguitate
    }
}
```

Utilitatea interfeţelor

- Definirea unor similarităţi între clase independente.
- Impunerea unor specificaţii: asigură că toate clasele care implementează o interfaţă pun la dispoziţie metodele specificate în interfaţă - de aici rezultă posibilitatea implementării unor clase prin mai multe modalităţi şi folosirea lor într-o manieră unitară;
- Definirea unor grupuri de constante
- Transmiterea metodelor ca parametri

- Crearea grupurilor de constante:

```
public interface Luni {
    int IAN=1, FEB=2, ..., DEC=12;
}
...
if (luna < Luni.DEC)
    luna ++;
else
    luna = Luni.IAN;
```

Transmiterea metodelor ca parametri

```
interface Functie {
    void executa(Nod u);
}
class Graf {
    void explorare(Functie f) {
            ...
            if (explorarea a ajuns in nodul v)  f.executa(v);
    }
}
//Definim diverse functii
class AfisareRo implements Functie {
    public void executa(Nod v) {
            System.out.println("Nodul curent este: " + v);
    }
}
class AfisareEn implements Functie {
    public void executa(Nod v) {
            System.out.println("Current node is: " + v);
    }
}
```

Transmiterea metodelor ca parametri (2)

```
public class TestCallBack {
    public static void main(String args[]) {
        Graf g = new Graf();
        Functie f1 = new AfisareRo();
        g.explorare(f1);
        Functie f2 = new AfisareEn();
        g.explorare(f2);
    /* sau mai simplu:
        g.explorare(new AfisareRo());
        g.explorare(new AfisareEn());
    */
    }
}
```

Interfaţa FilenameFilter

- folosită pentru a crea filtre pentru fişiere
- sunt primite ca argumente de metode care listează conţinutul unui director, cum ar fi metoda *list* a clasei File.
- putem spune că metoda *list* primeşte ca argument o altă funcţie care specifică dacă un fişier va fi returnat sau nu (criteriul de filtrare).
- Exemplu: Listarea fişierelor din directorul curent care au anumită extensie primită ca argument. Dacă nu se primeşte nici un argument , vor fi listate toate.

```
import java .io .*;
class Listare {
   public static void main ( String [] args ) {
        try {
                File director = new File (".");
                String [] listF ;
```

Interfaţa FilenameFilter: exemplu

```
    if ( args . length > 0)
        listF = director . list ( new Filtru ( args [0]) );
    else
        listF = director . list ();
    for (int i = 0; i < listF . length ; i ++)
        System . out. println ( listF [i]);
    } catch ( Exception e) {   e. printStackTrace (); }
  }

class Filtru implements FilenameFilter {
    String extensie ;
    Filtru ( String extensie ) {
        this . extensie = extensie ;
    }
    public boolean accept ( File dir , String nume ) {
        return ( nume . endsWith ("." + extensie ) );
    }
}}
```

Compararea obiectelor

Exemplu: Clasa Persoana (fără suport pentru comparare)

```
class Persoana {
    private int cod ;
    private String nume ;
    public Persoana ( int cod , String nume ) {
        this .cod = cod;
        this . nume = nume ;
    }
    public String toString () {
        return cod + " \t " + nume ;
    }
}
```

Exemplu: Sortarea unui vector de tip referinţă

```
class Sortare {
    public static void main ( String args []) {
        Persoana p[] = new Persoana [3];
        p[0] = new Persoana (3, " Ionescu ");
        p[1] = new Persoana (1, " Vasilescu ");
        p[2] = new Persoana (2, " Georgescu ");
        java . util . Arrays . sort (p);
        System . out. println (" Persoanele ordonate dupa cod:");
        for (int i=0; i<p. length ; i++) System . out. println (p[i]);
    }
}
```

Interfaţa Comparable

- Interfaţa Comparable impune o ordine totală asupra obiectelor unei clase ce o implementează. Această ordine se numeşte ordinea naturală a clasei şi este specificată prin intermediul metodei compareTo. Definiţia interfeţei este:

```
public interface Comparable {
    int compareTo(Object o);
}
```

- metoda compareTo trebuie să returneze:

• o valoare strict negativă: dacă obiectul curent (this) este mai mic decât obiectul primit ca argument;

• zero: dacă obiectul curent este egal cu obiectul primit ca argument;

• o valoare strict pozitivă: dacă obiectul curent este mai mare decât obiectul primit ca argument.

Interfaţa Comparable. Exemplu

Exemplu: Clasa Persoana cu suport pentru comparare

```
class Persoana implements Comparable {
    private int cod ;
    private String nume ;
    public Persoana ( int cod , String nume ) {
        this .cod = cod;
        this . nume = nume ;
    }
    public String toString () {
        return cod + " \t " + nume ;
    }
    public boolean equals ( Object o) {
        if (!( o instanceof Persoana ))  return false ;
        Persoana p = ( Persoana ) o;
        return (cod == p.cod) && ( nume . equals (p. nume ));
    }
    public int compareTo ( Object o) {
        if (o = = null ) throw new NullPointerException ();
        if (!( o instanceof Persoana ))
            throw new ClassCastException ("Nu pot compara !");
        Persoana p = ( Persoana ) o;
        return (cod - p.cod);
    }
}
```

Interfaţa Comparator

- In cazul în care dorim să sortăm elementele unui vector ce conţine referinţe după alt criteriu decât ordinea naturală a elementelor
- Interfaţa java.util.Comparator conţine metoda compare, care impune o ordine totală asupra elementelor unei colecţii.

  int compare(Object o1, Object o2);

```
class MyComp implements Comparator{
  public int compare ( Object o1 , Object o2) {
    Persoana p1 = ( Persoana )o1;
    Persoana p2 = ( Persoana )o2;
    return (p1. nume . compareTo (p2. nume ));
}
…
Arrays.sort(p, new MyComp());
```

Interfaţa Comparator

Exemplu: Sortarea unui vector folosind un comparator

```
import java . util .*;
class Sortare {
    public static void main ( String args []) {
            Persoana p[] = new Persoana [4];
            p[0] = new Persoana (3, " Ionescu ");
            p[1] = new Persoana (1, " Vasilescu ");
            p[2] = new Persoana (2, " Georgescu ");
            p[3] = new Persoana (4, " Popescu ");
            Arrays . sort (p, new Comparator () {
                        public int compare ( Object o1 , Object o2) {
                            Persoana p1 = ( Persoana )o1;
                            Persoana p2 = ( Persoana )o2;
                            return (p1. nume . compareTo (p2. nume ));
                        }
            } );
            System . out. println (" Persoanele ordonate dupa nume :");
            for (int i=0; i<p. length ; i++)
                        System . out. println (p[i]);
    }
}
```

Problema

- Să se definească o clasă SortedVector derivată din Vector, care să permită ordonarea după orice criteriu, specificat de utilizator la construirea unui obiect SortedVector. Clasa va conţine o variabilă de tip Comparator, iniţializată de un constructor cu argument de tip Comparator şi folosită de metoda Collections.sort.
- Să se definească o clasă Pair care conţine două date de tip Object, cu metodele equals şi toString redefinite.
- Să se scrie două clase Comparator pentru compararea obiectelor Pair dupa primul obiect, respectiv, comparare după al doilea obiect din pereche

- Să se scrie un program pentru crearea a doi vectori SortedVector de obiecte Pair, unul ordonat după primul obiect din pereche şi celălalt ordonat după al doilea obiect: vom adăuga obiecte Pair ce conţin un String şi un Integer.

Rezolvare 1

```
import java.util.*;
class SortedVector extends Vector{
    Comparator c;
    public SortedVector(Comparator c){
        this.c=c;
    }
    /*public SortedVector(){
        this(null);
    } */
    public boolean add(Object o){
        super.add(o);
        Collections.sort(this, c);
        return true;
    }
}
```

Rezolvare 2

```
class Pair {
    Object a,b;
    public Pair(Object a,Object b){
        this.a=a;
        this.b=b;
    }
    public boolean equals(Object o){
        if (o==null) return false;
        if(!(o instanceof Pair)) return false;
        Pair p=(Pair)o;
        return p.a.equals(a)&&p.b.equals(b);
    }
    public String toString(){
        return ""+a+" "+b;
    }
  }
```

Rezolvare 3

```
class Comp1 implements Comparator{
    public int compare(Object o1, Object o2){
        Pair p1=(Pair)o1;
        Pair p2=(Pair)o2;
        return ((Comparable)(p1.a)).compareTo(p2.a);
    }
}

class Comp2 implements Comparator{
    public int compare(Object o1, Object o2){
        Pair p1=(Pair)o1;
        Pair p2=(Pair)o2;
        return ((Comparable)(p1.b)).compareTo(p2.b);
    }
}
```

Rezolvare 4

```
class Test {
   public static void main(String[] args) {
    SortedVector v1=new SortedVector(new Comp1());
    SortedVector v2=new SortedVector(new Comp2());
    v1.add(new Pair("x",1));
    v1.add(new Pair("a",3));
    v2.add(new Pair("a",3));
    v2.add(new Pair("x",1));
    System.out.println(v1);
    System.out.println(v2);
    }
}
```